Análisis y Diseño de algoritmos

Práctica 2 — Multiplicación rápida de enteros: Algoritmo de Karatsuba

Alejandro Cifuentes Arroyave

EAFIT

Carlos Alberto Álvarez Henao

1 de noviembre de 2026

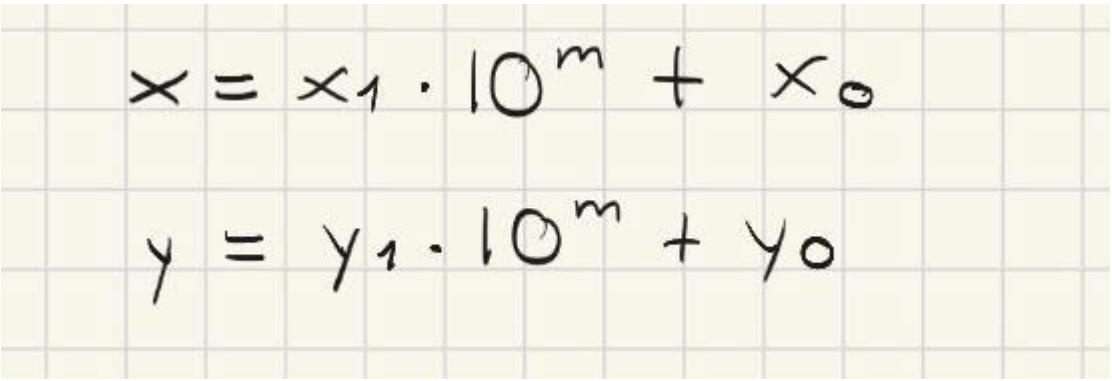
La deducción de la identidad de Karatsuba:

Para multiplicar dos enteros x e y de n dígitos en base B = 10, se aplica la estrategia de Divide y Vencerás particionando cada número en sus mitades alta y baja respecto a

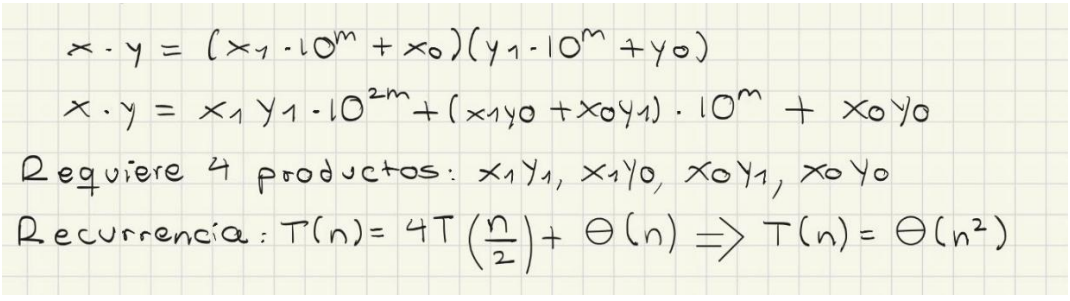
m = n/2: $x = x1 \cdot 10^m + x0$ y $y = y1 \cdot 10^m + y0$. La expansión algebraica directa de este producto conduce a cuatro multiplicaciones recursivas entre mitades de tamaño n/2, lo que genera una recurrencia $T(n) = 4T(n/2) + O(n) = O(n^2)$, la cual no supera la complejidad del algoritmo tradicional.

A continuación, se presenta la deducción formal que ideó Anatolii Karatsuba en 1960, donde mediante una manipulación algebraica se reduce el cálculo a únicamente tres productos recursivos (P1, P2, P3), recuperando el término central mediante restas y permitiendo una recomposición garantizada sin signos negativos:

1. **Partición de los operandos ($m = n/2$):**



2. **Multiplicación directa por Divide y Vencerás:**



3. **Identidad de Karatsuba (Definición de 3 productos):**

$$P_1 = x_1 \cdot y_1$$

$$P_2 = x_0 \cdot y_0$$

$$P_3 = (x_1 + x_0)(y_1 + y_0)$$

4. **Expansión algebraica de P_3 y cálculo del término central:**

$$P_3 = x_1 y_1 + x_1 y_0 + x_0 y_1 + x_0 y_0$$

$$P_3 = P_1 + (x_1 y_0 + x_0 y_1) + P_2$$

$$x_1 y_0 + x_0 y_1 = P_3 - P_1 - P_2$$

5. **Recomposición del producto final:**

$$x \cdot y = P_1 \cdot 10^{2m} + (P_3 - P_1 - P_2) \cdot 10^m + P_2$$

6. **Garantía de no negatividad (Orden de la resta):**

$$P_3 - P_1 = P_2 + x_1 y_0 + x_0 y_1 \geq P_2$$

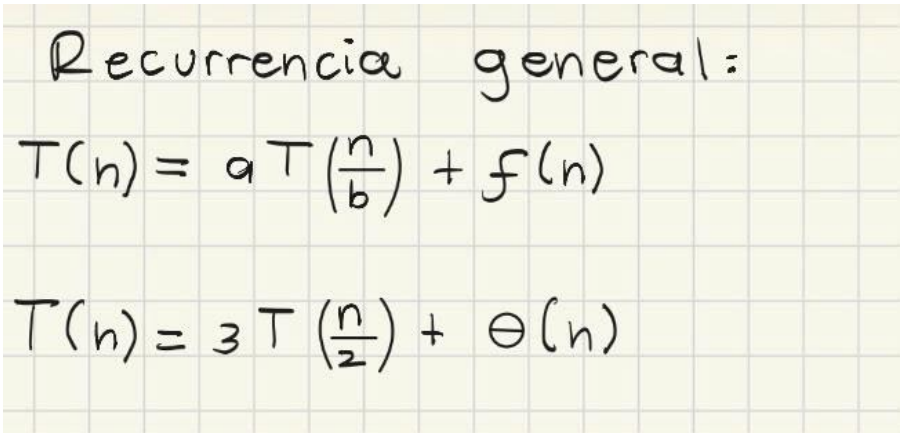
$$\Rightarrow (P_3 - P_1) - P_2 \geq 0$$

Recurrencia resultante: $T(n) = 3T\left(\frac{n}{2}\right) + \Theta(n)$

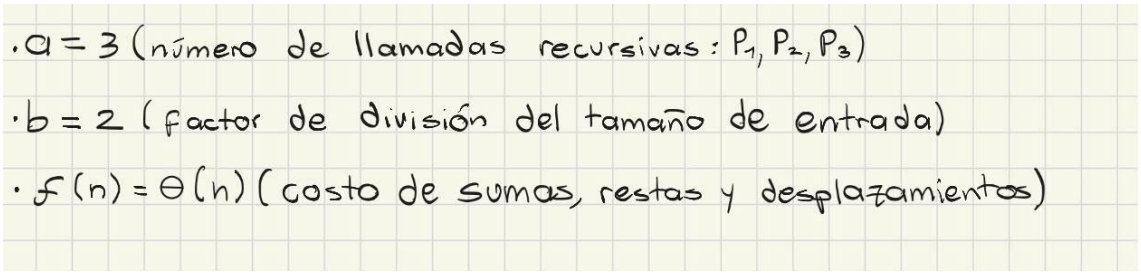
Resolución mediante el Teorema Maestro

A partir de la reducción a tres subproblemas de tamaño n/2 y el costo lineal O(n) asociado a las sumas, restas y desplazamientos de dígitos, se obtiene la relación de recurrencia T(n) = 3T(n/2) + O(n). A continuación, se presenta la aplicación formal del Teorema Maestro para determinar su cota asintótica:

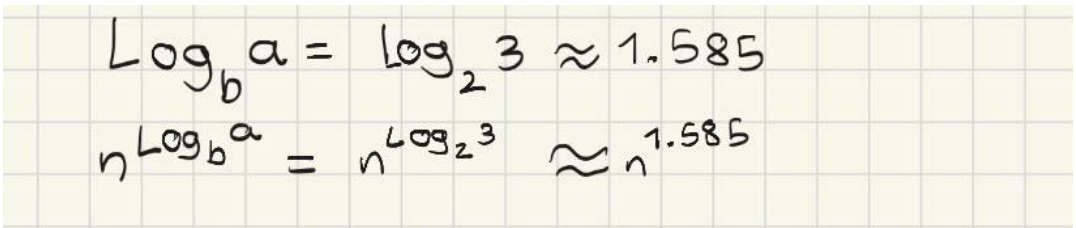
Recurrencia general:



1. **Identificación de parámetros:**



2. **Cálculo del exponente crítico (costo de las hojas):**



3. **Comparación entre $f(n)$y $n^{\log_b a}$:**

$f(n) = \Theta(n) = \Theta(n^1)$

Dado que $1 < \log_2 3$, existe $\epsilon = \log_2 3 - 1 \approx 0.585 > 0$ tal que:

$f(n) = O(n^{\log_2 3 - \epsilon}) = O(n^1)$

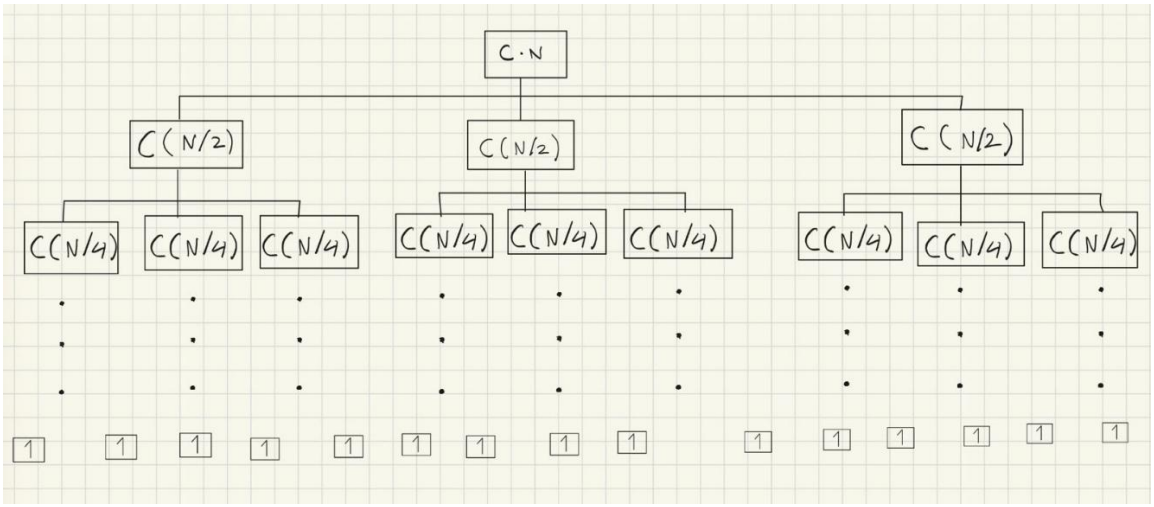
4. **Justificación del caso aplicable:** Como $f(n)$está acotada polinomialmente por debajo de $n^{\log_2 3}$por un factor n^{ϵ}, aplica el **Caso 1** del Teorema Maestro (el trabajo en las hojas domina asintóticamente al trabajo de combinación en la raíz).
5. **Conclusión:**

$$T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$$

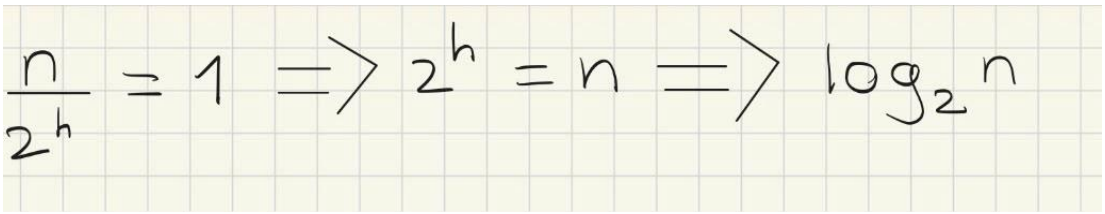
Demostración mediante el Árbol de Recurrencia

Para comprender a fondo la dinámica de la recursión, se desarrolla el árbol de recurrencia para T(n) = 3T(n/2) + c·n. Este método permite desglosar de manera explícita el número de subproblemas por nivel, el costo de combinación en cada etapa, la altura total del árbol y la acumulación del costo en las hojas, justificando rigurosamente de dónde surge el exponente $\log_2 3$:

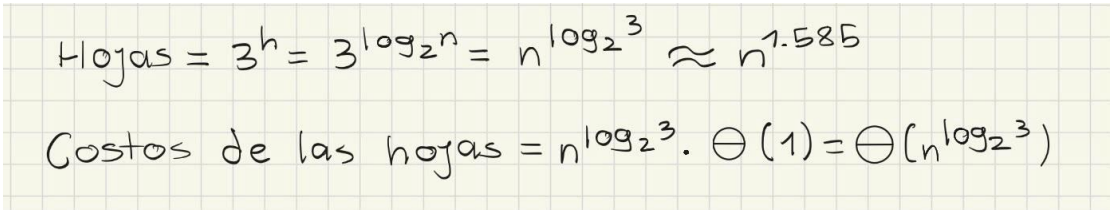
Esquema del Árbol:



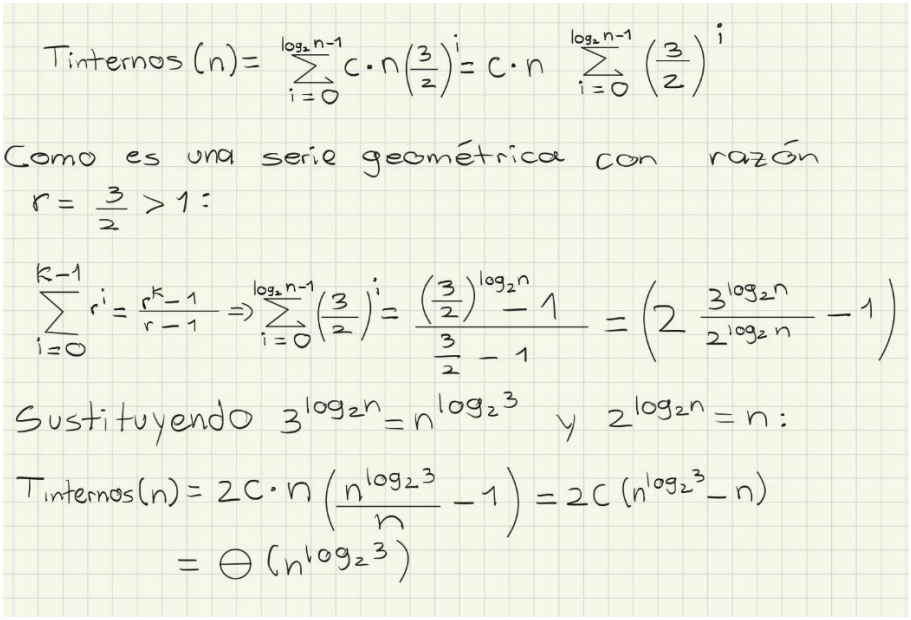
1. **Altura del árbol (h): La recursión se detiene en el caso base de 1 dígito:**



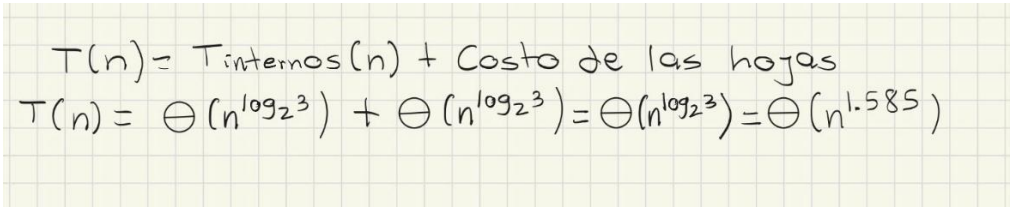
2. **Número de hojas y costo en la base: En el último nivel ($h = \log_2 n$):**



3. **Costo total de los niveles internos (Sumatoria): Sumamos el costo de combinación desde el nivel $i = 0$ hasta $h - 1$:**



4. **Costo total del algoritmo:**



5. Justificación del exponente $\log_2 3$: El factor de ramificación ($a = 3$) supera al factor de reducción ($b = 2$), dando una razón $r = \frac{3}{2} > 1$. Como el costo crece geométricamente hacia abajo, el tiempo total queda dominado por el nivel de las hojas, cuya cantidad exacta es $n^{\log_2 3}$.

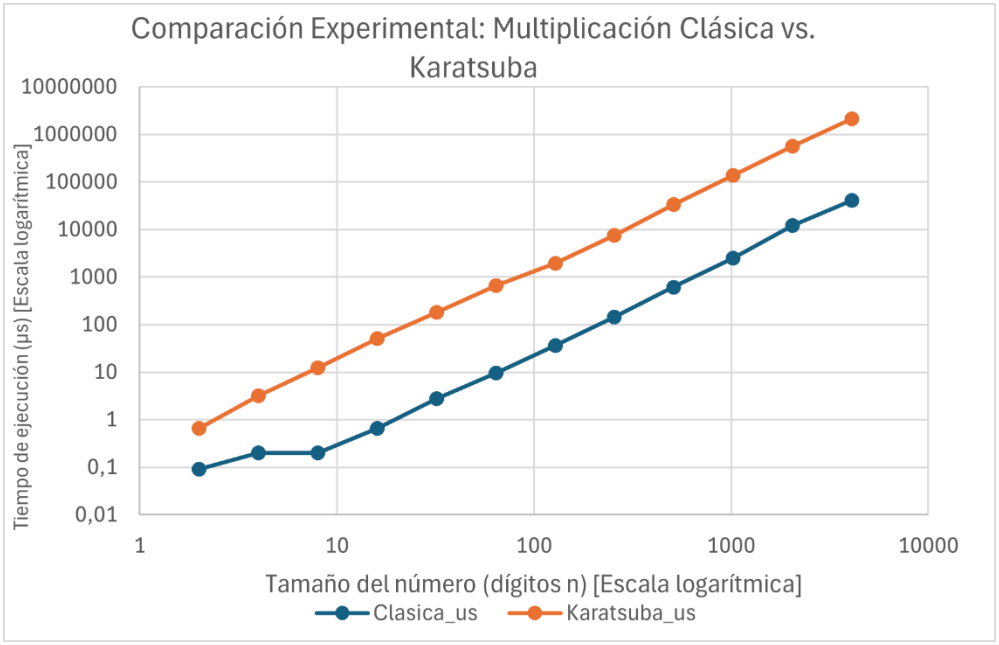
Análisis Experimental

Montaje experimental

Las pruebas se ejecutaron en C++17 compilado con optimización -O2. Para la medición se utilizó std::chrono::steady_clock, promediando múltiples repeticiones por cada tamaño n (desde 200 repeticiones para n ≤ 32 hasta 3 repeticiones para n = 4096) con el fin de superar la resolución del reloj en tamaños pequeños. Además, el resultado de cada multiplicación se acumuló en una variable 'volatile' para evitar que el compilador eliminara el cálculo por optimización.

Tabla de tiempos y gráfica

n	repeticiones	tiempo_clasica_us	tiempo_karatsuba_us
2	200	0.13	1.03
4	200	0.20	3.23
8	200	0.38	13.97
16	200	0.66	51.90
32	200	2.51	180.71
64	30	14.54	701.41
128	30	41.22	2018.19
256	30	151.64	7738.94
512	5	611.86	34161.90
1024	5	2315.96	138055.46
2048	3	9990.33	540634.17
4096	3	38409.67	2096518.93



Interpretación del punto de cruce y rendimiento real

En el rango medido (de 2 a 4096 dígitos) no se presentó punto de cruce, la multiplicación clásica fue más rápida que Karatsuba en todas las instancias evaluadas. Este comportamiento se explica al distinguir la complejidad asintótica del rendimiento real en máquina:

Complejidad asintótica vs. Constantes ocultas: Aunque Karatsuba tiene una mejor complejidad teórica (O(n^1.585) frente a $O(n^2)$), la constante oculta de Karatsuba es significativamente mayor. Al tener un caso base de 1 dígito, para n = 4096 se generan 3^{12} = 531,441 llamadas recursivas, creando una sobrecarga masiva en la pila de llamadas (call stack).

Copias y asignaciones de memoria: En cada llamada recursiva de Karatsuba se crean y destruyen múltiples objetos std::string en memoria dinámica (para x1, x0, y1, y0, sumas, restas y desplazamientos). Esta constante solicitud de memoria degrada la localidad de referencia en la memoria caché del procesador. En contraste, la multiplicación clásica solo realiza una asignación de memoria al inicio y opera mediante dos bucles iterativos simples y directos**.**

Por estas razones, para tamaños pequeños y medianos el método clásico resulta mucho más eficiente en la práctica, desplazando el punto de cruce teórico hacia tamaños de entrada superiores a 4096 dígitos.

Conclusiones y Dificultades Encontradas

Conclusiones:

Se comprobó experimentalmente que un algoritmo con mejor complejidad asintótica teórica (como Karatsuba con O(n^1.585)) no siempre es más rápido en la práctica. Para números pequeños y medianos (hasta 4096 dígitos), la multiplicación clásica fue superior debido al costo de tantas llamadas recursivas y la creación constante de cadenas en memoria.

La multiplicación clásica demostró un comportamiento cuadrático exacto: en la tabla de tiempos se observa claramente que cada vez que se duplica la cantidad de dígitos,

el tiempo de ejecución se multiplica prácticamente por 4, cumpliendo con precisión la teoría de $O(n^2)$.

Dificultades encontradas:

Manejo de las restas sin signo en Karatsuba: Para no complicar el código implementando números negativos, fue necesario seguir al pie de la letra el orden de resta (P3 - P1) - P2 propuesto en la guía, asegurando que cada resta intermedia fuera siempre positiva.

Margen de error en las mediciones por el sistema operativo: Al medir números pequeños, los tiempos eran de fracciones de microsegundo, por lo que procesos en segundo plano de Windows o la propia resolución del reloj del sistema podían generar variaciones en los resultados.

Declaración de uso de la IA:

Se utilizo la herramienta Gemini(Octubre 31 - Noviembre 1, 2026) para aclarar conceptos como las deducciones, las recurrencias resueltas, revisión de posibles mejoras al código, también, para la redacción adecuada del texto, de decisiones de diseño como el manejo de las restas sin signo en Karatsuba, la elaboración del README.md y revisión de los criterios necesarios de la práctica. Todo el código fuente, los experimentos y el análisis presentado fueron ejecutados, comprendidos y validados en su totalidad por el autor.